Hypergraphes acycliques et résolution de contraintes

Nathan TILLIER

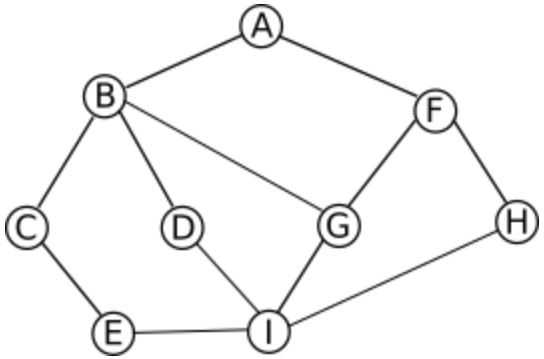
Sommaire

1. Introduction
2. Algorithmes
3. Implémentation
4. Résultats expérimentaux
5. Conclusion

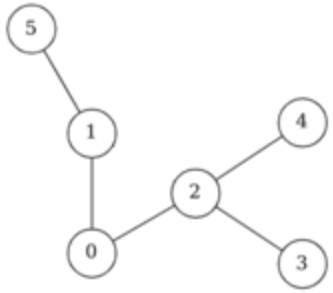
Introduction

Graphes et acyclicité

Graphe cyclique



Graphe acyclique



Liste d'adjacence

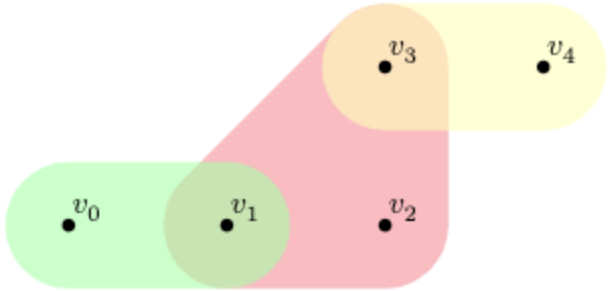
```
A : B, F
B : A, C, D, G
C : B, E
D : B, I
...
```

Liste d'adjacence

```
5 : 1
1 : 0, 5
0 : 1, 2
2 : 0, 3, 4
...
```

Hypergraphes

Hypergraphe



Liste d'adjacence

```
Vert : 0, 1
Rouge : 1, 2, 3
Jaune : 4
```

Définition

Soit $X = \{v_0, v_1, \ldots, v_n\}$ un ensemble fini.
Un hypergraphe sur X est une famille $H = (E_0, E_2, \ldots, E_p)$ de parties de X vérifiant

$$E_i \neq \emptyset \text{ pour } i = 0, 1, \ldots, p$$

Et

$$\bigcup_{i=0}^{p} E_i = X$$

Arbres de jointure

Un arbre de jointure est quelque chose.

Base de données

On va s'intéresser au schéma suivant :

Communes

Identifiant	Région	Département	Nom
75056	11	75	Paris
29019	53	29	Brest
69123	84	69	Lyon

Régions et départements

Identifiant	Cheflieu	Nom
11	75056	Île-de-France
53	35238	Bretagne
84	69123	Auvergne-Rhône-Alpes

Identifiant	Région	Nom
75	11	Paris
29	53	Finistère
69	84	Rhône

© Hypergraphes acycliques

Requête conjonctive acyclique

Utilisation des hypergraphes

Algorithmes

Réduction Graham-Yu-Ozsoyoglu

Algorithme de Yannakakis

Implémentation

Structure d'hypergraphe

On se munit de fonctions auxiliaires pour manipuler les hypergraphes.

```c
typedef struct arete {
    int cardinal;
    int* sommets;
} arete;

typedef struct hyper {
    int ordre;
    int nb_aretes;
    arete** aretes;
} hyper;
```

```c
arete* creer_arete(int cardinal);
void liberer_arete(arete* a);
arete* copier_arete(arete* a);
void augmenter_arete(arete* a, int sommet);
void reduire_arete(arete* a, int sommet);
bool dans_arete(arete* a, int sommet);

hyper* creer_hyper(int ordre, bool strict);
bool est_valide(hyper* h);
void ajouter_arete(hyper* h, arete* a);
void enlever_arete(hyper* h, int indice);
```

Résultats expérimentaux

Résultats expérimentaux